

DOCUMENTAȚIE PROIECT ISP

Aplicație pentru Gestionarea Voluntarilor la ONG

Nume: Cismaru Adrian | Grupa: 332AA

Descriere generală

Proiectul reprezintă un sistem software dezvoltat în Java, destinat administrării activităților unui ONG. Aplicația permite gestionarea a trei tipuri principale de utilizatori:

- Administratori — creează activități și rapoarte
- Voluntari — se pot înscrie la activități și pot genera adeverințe cu istoricul de voluntariat
- Donatori — pot realiza donații pentru a susține ONG-ul

Proiectul este organizat în două pachete distincte (models și tests). Funcționalitățile critice sunt testate automatizat cu JUnit, iar pentru exportul rapoartelor și adeverințelor a fost integrată biblioteca externă iTextPDF.

1. Partea I — Lucru în echipă

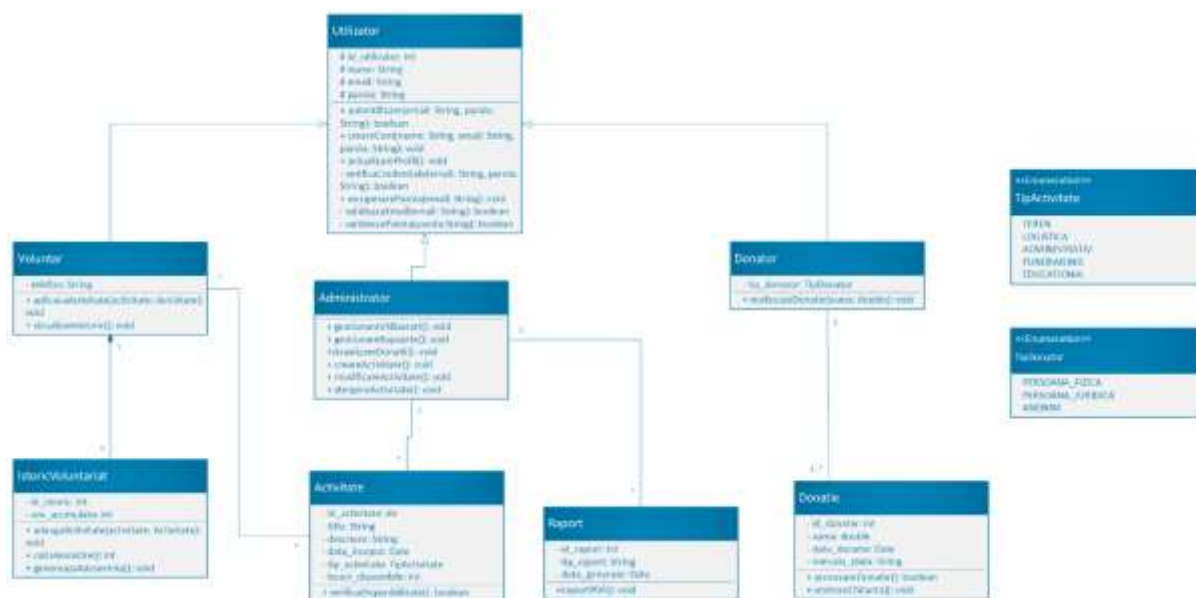
1.1 Diagrama de clase

Descrierea modelului

Diagrama de clase modelează structura sistemului și conține cele trei tipuri de relații obligatorii studiate:

- Generalizare (Moștenire):** Clasa abstractă *Utilizator* este extinsă de clasele derivate *Administrator*, *Voluntar* și *Donator*. Toate moștenesc atributele de bază (*id_utilizator*, *nume*, *email*, *parola*) și comportamentele comune (autentificare, creare cont).
- Compoziție:** Există o relație strânsă între *Voluntar* și *IstoricVoluntariat* (1 la 1). Istoricul este instanțiat direct în constructorul voluntarului, astfel ciclul de viață al istoricului depinde de cel al voluntarului.

- **Asociere:** Clasa *Administrator* este asociată cu gestionarea claselor *Activitate* și *Raport*. Relații de asociere există de asemenea între *Donator* și *Donatie* (1 la 1..*), dar și între *Voluntar* și *Activitate* (un voluntar poate participa la mai multe activități, iar o activitate are o listă de voluntari).
- **Enumerări:** Sunt folosite enums precum *TipActivitate* (TEREN, LOGISTICA, ADMINISTRATIV, FUNDRAISING, EDUCATIONAL) și *TipDonator* (PERSOANA_FIZICA, PERSOANA_JURIDICA, ANONIM).

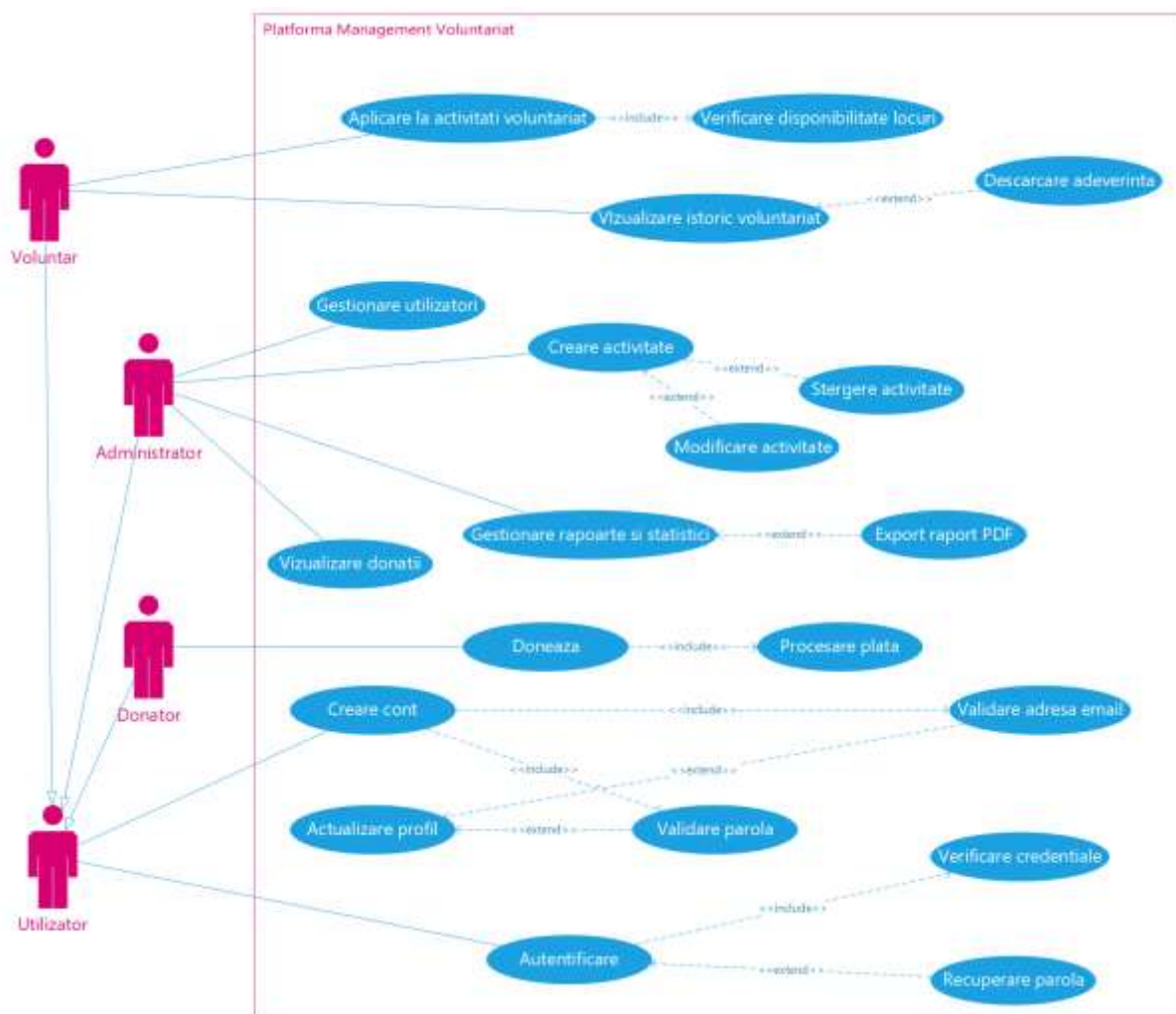


1.2 Diagrama cazurilor de utilizare

Descrierea modelului

Diagrama surprinde interacțiunea actorilor cu sistemul:

- **Actori:** Utilizator (actor general), Voluntar, Administrator, Donator.
- **Cazuri de utilizare principale:** Autentificare, Creare cont (comune tuturor utilizatorilor).
 - Administratorul are acces la: Creare / Modificare / Ștergere activitate, Gestionare utilizatori și Gestionare rapoarte.
 - Voluntarul poate: Aplica la activități, Vizualiza și descărca adeverința.
 - Donatorul poate: Realiza donații.
- **Relații <<include>> și <<extend>>:**
 - Include: «Donează» include «Procesare plată»; «Aplicare la activități voluntariat» include obligatoriu «Verificare disponibilitate locuri».
 - Extend: «Creare activitate» poate fi extinsă opțional de «Modificare activitate» sau «Ștergere activitate»; «Vizualizare istoric voluntariat» poate fi extins de «Descărcare adeverință».



2.1 Diagrama de Activități

Descrierea modelului

Această diagramă detaliază cazul de utilizare „**Realizare donație**” și modelează interacțiunea dintre Donator și Sistem în procesul complet de efectuare a unei donații. Diagrama este organizată în două swimlane-uri distincte:

- **Donator** — inițiază fluxul prin solicitarea donației și introduce datele (suma și metoda de plată).
- **Sistem** — afișează formularul, validează datele, procesează donația și emite confirmare.

Fluxul principal:

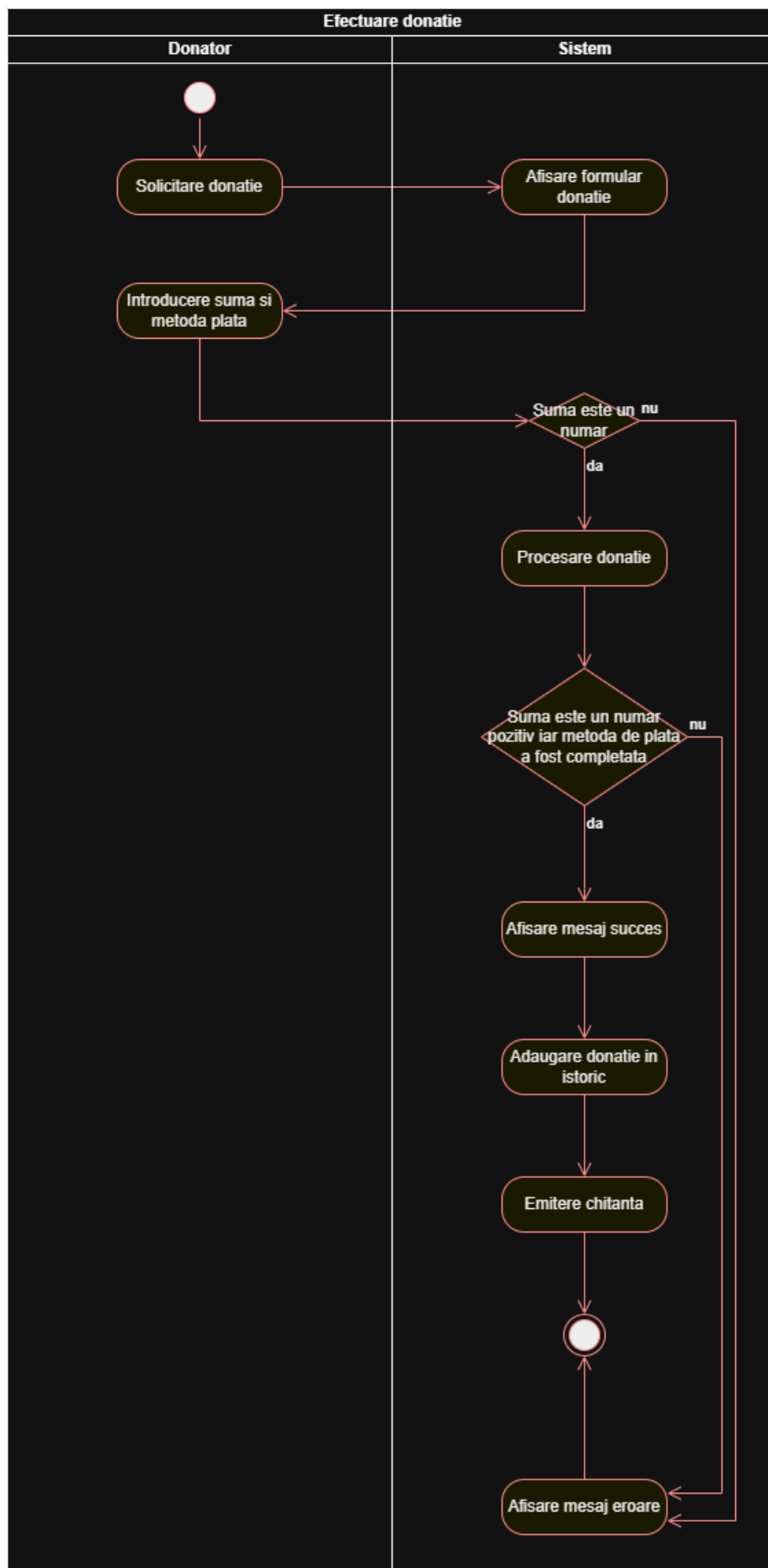
- Donatorul solicita efectuarea unei donatii; Sistemul afiseaza formularul corespunzator.
- Donatorul introduce suma si metoda de plata dorita.
- Sistemul verifica daca suma introdusa este un numar valid.
- Daca suma este valida: Sistemul proceseaza donatia si verifica daca suma este pozitiva si metoda de plata a fost completata.
- Daca ambele conditii sunt indeplinite: Sistemul afiseaza un mesaj de succes, adauga donatia in istoricul donatorului si emite o chitanta.

Noduri de decizie:

- **Decizia 1 — „Suma este un număr?”:** Dacă nu, fluxul se ramifică direct către afișarea unui mesaj de eroare, finalizând fluxul alternativ.
- **Decizia 2 — „Suma pozitivă și metoda validată?”:** Dacă metoda de plată nu este validată sau suma este negativă sau zero, fluxul se ramifică spre mesajul de eroare.

Fluxul alternativ (eroare):

Oricare dintre cele două condiții invalide conduc la afișarea unui mesaj de eroare în swimlane-ul Donator, după care fluxul se încheie.



2.2 Diagrama de Stare

Descrierea modelului

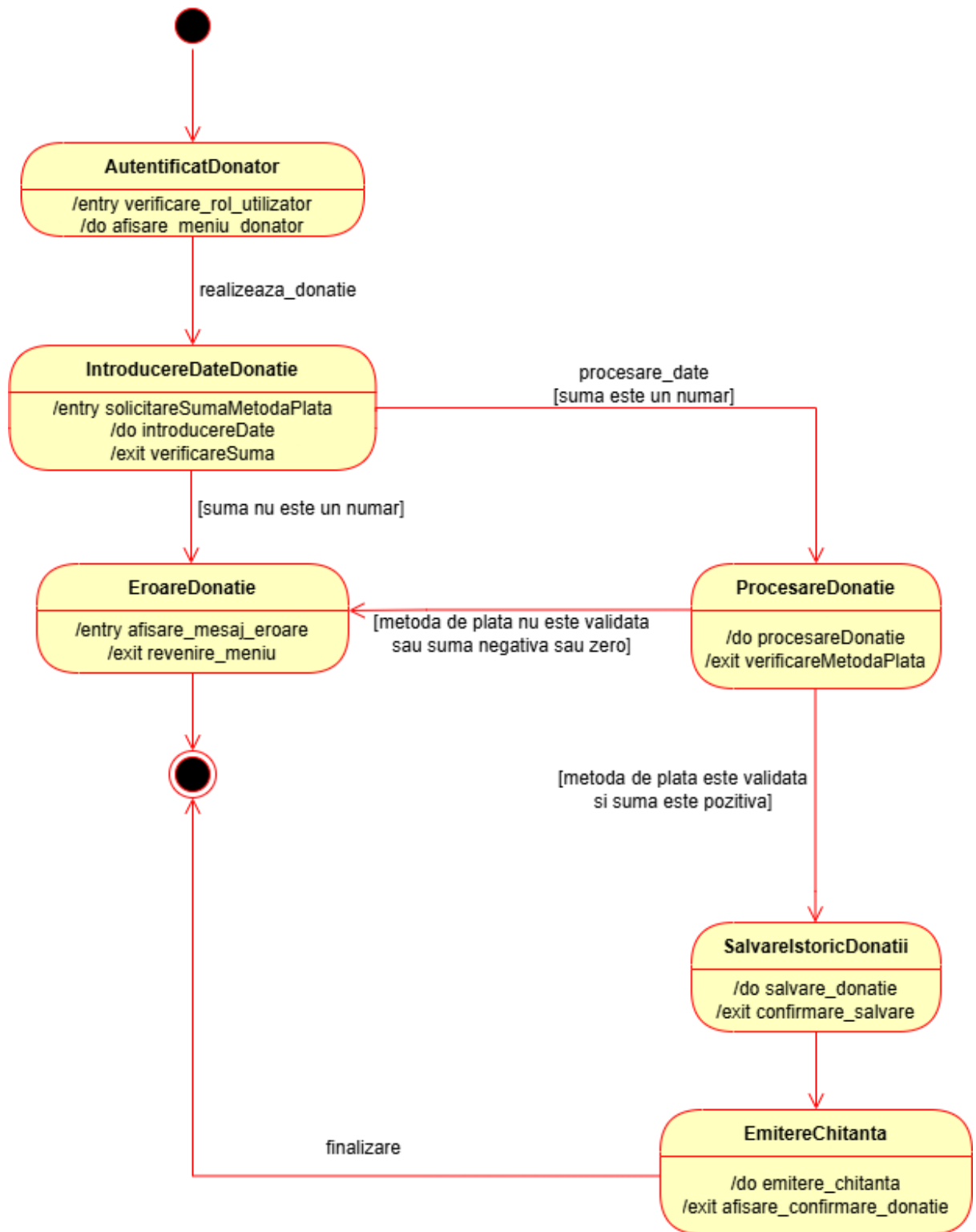
Această diagramă de stare modelează ciclul de viață al obiectului *Donatie* din perspectiva tranzacției efectuate de un donator. Fiecare stare include acțiunile interne (entry, do, exit) extrase direct din implementarea Java.

Stările obiectului:

- **AutentificatDonator:** /entry verificare_rol_utilizator; /do afisare_meniu_donator — Donatorul este autentificat si are acces la meniul specific rolului sau.
- **IntroducereDateDonatie:** /entry solicitareSumaMetodaPlata; /do introducereDate; /exit verificareSuma — Donatorul completeaza suma si metoda de plata; la iesire se verifica daca suma este un numar.
- **ProcesareDonatie:** /do procesareDonatie; /exit verificareMetodaPlata — Sistemul proceseaza intern datele si verifica la iesire daca metoda de plata este valida si suma este pozitiva.
- **EroareDonatie:** /entry afisare_mesaj_eroare; /exit revenire_meniu — Stare alternativa activata cand suma nu este numar, sau metoda este invalida, sau suma este negativa/zero. Fluxul se incheie dupa afisarea erorii.
- **SalvareIstoricDonatii:** /do salvare_donatie; /exit confirmare_salvare — Donatia este persistata in istoricul donatorului.
- **EmitereChitanta:** /do emitere_chitanta; /exit afisare_confirmare_donatie — Sistemul genereaza si afiseaza chitanta, finalizand fluxul cu succes.

Tranzițiile:

- *AutentificatDonator -> IntroducereDateDonatie : realizeaza_donatie*
- *IntroducereDateDonatie -> ProcesareDonatie : [suma este un numar]*
- *IntroducereDateDonatie -> EroareDonatie : [suma nu este un numar]*
- *ProcesareDonatie -> SalvareIstoricDonatii : [metoda de plata este validata si suma este pozitiva]*
- *ProcesareDonatie -> EroareDonatie : [metoda de plata nu este validata sau suma negativa sau zero]*
- *SalvareIstoricDonatii -> EmitereChitanta*
- *EmitereChitanta -> [end] : finalizare*
- *EroareDonatie -> [end]*



Nota: Ambele diagrame sunt bazate direct pe implementarea clasei *Donatie.java* si a clasei *Donator.java* din proiect, respectand logica de validare implementata in metoda *procesareDonatie()* si fluxul din *Main.java*.